TP 7 – Explorateur d'API REST

TP global 3/4 – fetch, chargement, erreurs

Pr. Ahmed Bentajer

Objectifs

- Récupérer des données depuis une vraie API avec `fetch`
- Gérer les trois états d'un chargement réseau : en cours, réussi, en erreur
- Enchaîner `async`/`await` avec un hook de cycle de vie

Cahier des charges

Une petite application « explorateur de pays », à partir de l'API publique REST Countries (déjà citée dans les notes de votre support, avec JSONPlaceholder) :

- une page liste qui charge tous les pays au montage, avec une recherche par nom ;
- une page détail par pays (route `/pays/:code`), chargée séparément depuis l'API.

Le routeur, `CountryCard.vue` et toute la structure des pages sont déjà en place – il ne manque que les appels réseau et le filtre de recherche. Aucune clé d'API n'est nécessaire.

`npm install && npm run dev` pour démarrer.

Pour info

La forme générale d'un chargement réseau dans un composant Options API est toujours la même, et vous la retrouverez telle quelle dans énormément de code Vue réel :

```
async chargerDonnees() {
  this.loading = true;
  this.error = null;
  try {
    const reponse = await fetch(URL);
    if (!reponse.ok) throw new Error(`Erreur ${reponse.status}`);
    this.donnees = await reponse.json();
  } catch (e) {
    this.error = "Impossible de charger les donnees.";
  } finally {
    this.loading = false;
  }
}
```

1 Liste des pays

Dans `CountryListView.vue`, complétez `chargerPays()` : elle doit appeler `fetch(API_URL)` et remplir `this.countries`, en suivant le schéma ci-dessus pour `loading` et `error`. Complétez ensuite `paysAffiches` pour filtrer `this.countries` selon `this.recherche` (comparaison sur `country.name.common`, insensible à la casse).

Piège classique

`fetch()` ne rejette **que** sur une panne réseau (pas de connexion, DNS...). Une réponse `404` ou `500` du serveur est une réponse `fetch` « réussie » du point de vue de la promesse – `reponse.ok` sera simplement `false`. Sans le `if (!reponse.ok) throw ...`, une erreur serveur passerait inaperçue et `reponse.json()` planterait plus loin avec un message bien moins clair.

2 Fiche détail

Dans `CountryDetailView.vue`, complétez `chargerPays()` sur le même principe, avec l'URL `https://restcountr` où `code` vient de `this.$route.params.code`. Attention : selon la version de l'API, la réponse peut être un objet unique ou un tableau contenant un seul objet – gérez les deux cas avant d'affecter `this.country`.

Bonus

Le chargement de la fiche détail ne se relance qu'au montage du composant (`mounted`). Si un jour vous ajoutez un lien d'un pays vers un autre pays sur la même vue (par exemple des pays voisins), Vue Router **réutilise** le même composant au lieu de le démonter/remonter – `mounted` ne se redéclenchera donc pas. Renseignez-vous sur `watch` appliqué à `this.$route.params.code` pour recharger les données dans ce cas.

Dernier TP global : la migration de l'un de ces projets vers la Composition API et Pinia, en préparation directe du mini-projet.